Capron Dylan

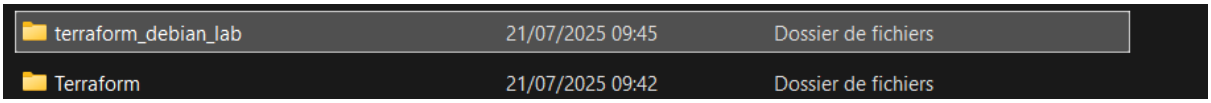
Terraform

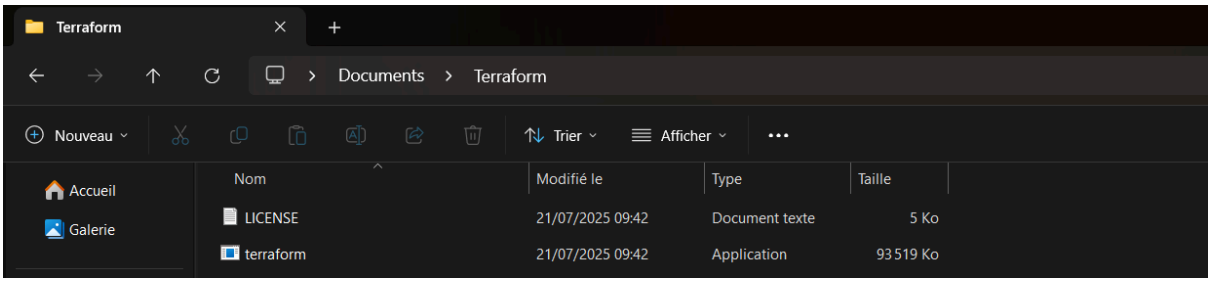
Préparation de l'environnement

```
login as: t
t@192.168.159.167's password:
Linux debian 6.1.0-37-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.140-1 (2025-05-22)
 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Jul 21 09:15:23 2025
t@debian:~$ sudo whoami
[sudo] Mot de passe de t :
root
t@debian:~$
```

C:\Users\dylan\OneDrive\Documents\Virtual Machines\Terraform\Terraform.vmx

Installation de Terraform






```
PS C:\Users\dylan> terraform --version
Terraform v1.12.2
on windows_amd64
PS C:\Users\dylan>
```

Premier déploiement de VM Debian 12

Main.tf :

```
# main.tf

terraform {
  required_providers {
    null = {
      source = "hashicorp/null"
      version = "~> 3.0"
    }
  }
  required_version = ">= 1.0.0"
}

# --- Resource to Clone VM ---
# null_resource pour  commandes (curl)
# via 'local-exec' provisioner
resource "null_resource" "clone_vm" {

  triggers = {
    template_vm_id   = var.template_vm_id
    new_vm_name      = var.new_vm_name
    vm_clone_path    = var.vm_clone_path
  }

  provisioner "local-exec" {
    # Environment variables
    environment = {
      TF_TEMPLATE_VM_ID  = var.template_vm_id
      TF_NEW_VM_NAME     = var.new_vm_name
      TF_VM_CLONE_PATH   = var.vm_clone_path
```

```
    TF_VMREST_HOST     = var.vmrest_host
    TF_VMREST_PORT     = var.vmrest_port
    TF_VMREST_USER     = var.vmrest_user
    TF_VMREST_PASSWORD = var.vmrest_password
  }

#    command = trimspace(<<EOT
#      bash -c "urlencode() { python3 -c 'import urllib.parse, sys;
print(urllib.parse.quote_plus(sys.stdin.read().rstrip(\"\\n\")))' <<<
\"\$1\"; }; echo \"DEBUG: Attempting to clone VM...\"; echo \"DEBUG:
Template ID (parentId): \$TF_TEMPLATE_VM_ID\"; echo \"DEBUG:
New VM Name: \$TF_NEW_VM_NAME\"; echo \"DEBUG: Target URL:
http://\$TF_VMREST_HOST:\$TF_VMREST_PORT/api/vms\"; echo
\"DEBUG: Data Payload: { \\\"name\\\":
\\\"\$TF_NEW_VM_NAME\\\", \\\"parentId\\\":
\\\"\$TF_TEMPLATE_VM_ID\\\" }\"; CLONE_RESPONSE=\$(curl -v
-X POST -H \"Content-Type: application/vnd.vmware.vmw.rest-v1+json\"
-u \"\$TF_VMREST_USER:\$TF_VMREST_PASSWORD\"
\"http://\$TF_VMREST_HOST:\$TF_VMREST_PORT/api/vms\" -d
'{\"name\":\"'\$TF_NEW_VM_NAME'\",
\"parentId\":\"'\$TF_TEMPLATE_VM_ID'\"}' 2>&1); echo \"DEBUG:
Curl command exited with status \$?.\"; echo \"DEBUG: Full CURL
Response/Errors:\"; echo \"\$CLONE_RESPONSE\"; echo \"DEBUG: End
of CURL Response.\"; JSON_PAYLOAD=\$(echo
\"\$CLONE_RESPONSE\" | awk '/^[{[]/ { p=1 } p; /^[}\\]]/ { p=0 }'); if
echo \"\$CLONE_RESPONSE\" | grep -q \"^< HTTP/1.[01] 20[01]\"; then
NEW_VM_ID=\$(echo \"\$JSON_PAYLOAD\" | jq -r '.id' 2>/dev/null);
if [ -z \"\$NEW_VM_ID\" ] || [ \"\$(echo \"\$JSON_PAYLOAD\" | jq
-r '.Message' 2>/dev/null)\" != \"null\" ]; then echo \"Error: vmrest clone
API returned an error or ID missing despite HTTP success.\"; echo \"Full
vmrest API JSON Response: \$JSON_PAYLOAD\"; exit 1; fi; else echo
\"Error: HTTP request did not return 2xx success status.\"; echo \"Full
CURL Response/Errors: \\n\$CLONE_RESPONSE\"; exit 1; fi; echo
```

```
\"Cloned VM ID: \$NEW_VM_ID\"; echo \"\$NEW_VM_ID\" >
\"/tmp/terraform_cloned_vm_id_\$TF_NEW_VM_NAME.txt\";"
#   EOT
#   )

   command = trimspace(<<EOT
     bash -c "urlencode() { python3 -c 'import urllib.parse, sys;
print(urllib.parse.quote_plus(sys.stdin.read().rstrip(\"\\n\")))' <<<
\"\$1\"; }; echo \"DEBUG: Attempting to clone VM...\"; echo \"DEBUG:
Template ID (parentId): \$TF_TEMPLATE_VM_ID\"; echo \"DEBUG:
New VM Name: \$TF_NEW_VM_NAME\"; echo \"DEBUG: Target URL:
http://\$TF_VMREST_HOST:\$TF_VMREST_PORT/api/vms\"; echo
\"DEBUG: Data Payload: { \\\"name\\\":
\\\"\$TF_NEW_VM_NAME\\\", \\\"parentId\\\":
\\\"\$TF_TEMPLATE_VM_ID\\\" }\"; CLONE_RESPONSE=\$(curl -v
-X POST -H \"Content-Type: application/vnd.vmware.vmw.rest-v1+json\"
-u \"\$TF_VMREST_USER:\$TF_VMREST_PASSWORD\"
\"http://\$TF_VMREST_HOST:\$TF_VMREST_PORT/api/vms\" -d
'{\"name\":\"'\$TF_NEW_VM_NAME'\",
\"parentId\":\"'\$TF_TEMPLATE_VM_ID'\"}' 2>&1); echo \"DEBUG:
Curl command exited with status \$?.\"; echo \"DEBUG: Full CURL
Response/Errors:\"; echo \"\$CLONE_RESPONSE\"; echo \"DEBUG: End
of CURL Response.\"; JSON_PAYLOAD=\$(echo
\"\$CLONE_RESPONSE\" | jq -c '.' | tail -1); if echo
\"\$CLONE_RESPONSE\" | grep -q \"^< HTTP/1.[01] 20[01]\"; then
NEW_VM_ID=\$(echo \"\$JSON_PAYLOAD\" | jq -r '.id' 2>/dev/null);
if [ -z \"\$NEW_VM_ID\" ] || [ \"\$(echo \"\$JSON_PAYLOAD\" | jq
-r '.Message' 2>/dev/null)\" != \"null\" ]; then echo \"Error: vmrest clone
API returned an error or ID missing despite HTTP success.\"; echo \"Full
vmrest API JSON Response: \$JSON_PAYLOAD\"; exit 1; fi; else echo
\"Error: HTTP request did not return 2xx success status.\"; echo \"Full
CURL Response/Errors: \\n\$CLONE_RESPONSE\"; exit 1; fi; echo
```

```
\"Cloned VM ID: \$NEW_VM_ID\"; echo \"\$NEW_VM_ID\" >
\"/tmp/terraform_cloned_vm_id_\$TF_NEW_VM_NAME.txt\";"
   EOT
   )

   # on_failure = continue pour debug
   on_failure = continue
 }
}

# terraform.tfvars
vmrest_user      = "dylan"
vmrest_password  = "Quiqui06*"
template_vm_id   = "T85S362F8IKU8OGPFM8V8NRJ5P23S7QH"
new_vm_name      = "CloneVM_Terraform"
vm_clone_path    = "C:/Users/dylan/Documents/Virtual
Machines/CloneVM_Terraform/CloneVM_Terraform.vmx"
```

```
PS C:\Users\dylan\OneDrive\Documents\terraform_debian_lab> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding elsudano/vmworkstation versions matching "2.0.1"...
- Installing elsudano/vmworkstation v2.0.1...
- Installed elsudano/vmworkstation v2.0.1 (self-signed, key ID 25B4C7DDA0B6F683)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://developer.hashicorp.com/terraform/cli/plugins/signing
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\dylan\OneDrive\Documents\terraform_debian_lab>
```

```
PS C:\Users\dylan\OneDrive\Documents\terraform_debian_lab> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # vmworkstation_virtual_machine.debian12 will be created
  + resource "vmworkstation_virtual_machine" "debian12" {
      + id         = (known after apply)
      + ip         = "0.0.0.0/0"
      + memory     = 4096
      + path       = "C:/Users/dylan/OneDrive/Documents/Virtual Machines/Terraform/Debian12-from-Terraform.vmx"
      + processors = 2
      + sourceid   = "C:/Users/dylan/OneDrive/Documents/Virtual Machines/Terraform/Terraform.vmx"
    }

Plan: 1 to add, 0 to change, 0 to destroy.

─────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if
you run "terraform apply" now.
```

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

null_resource.clone_vm: Creating...
null_resource.clone_vm: Provisioning with 'local-exec'...
null_resource.clone_vm (local-exec): (output suppressed due to sensitive value in config)
null_resource.clone_vm (local-exec): (output suppressed due to sensitive value in config)
null_resource.clone_vm: Creation complete after 0s [id=1566652168931883879]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

actual_cloned_vm_id = "ID not found or clone failed"
cloned_vm_name = "CloneVM_Terraform"
cloned_vm_path = "C:/Users/dylan/OneDrive/Documents/terraform_debian_lab/CloneVM_Terraform.vmx"
```

Amélioration et gestion du cycle de vie

```
# terraform.tfvars
vmrest_user      = "dylan"
vmrest_password  = "Quiqui06*"
template_vm_id   = "PFIGP9I3POIFRDU98CA8VRR7BD8LE893" #  ID => vmrest /api/vms
new_vm_name      = "CloneVM_Terraform"
vm_clone_path    = "C:/Users/dylan/OneDrive/Documents/terraform_debian_lab/CloneVM_Terraform.vmx"
```

terraform init

terraform plan

terraform apply

```
PS C:\Users\dylan\Onedrive\Documents\terraform_debian_lab> terraform destroy
null_resource.clone_vm: Refreshing state... [id=4774677232729209 33]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with th
following symbols:
  - destroy

Terraform will perform the following actions:

  # null_resource.clone_vm will be destroyed
  - resource "null_resource" "clone_vm" {
      - id       = "4774677232729209 33" -> null
      - triggers = {
          - "new_vm_name"    = "CloneVM_Terraform"
          - "template_vm_id" = "PFIGP9I3POIFRDU98CA8VRR7BD8LE893"
          - "vm_clone_path"  = "C:/Users/dylan/OneDrive/Documents/terraform_debian_lab/CloneVM_Terraform.vmx"
        } -> null
    }

Plan: 0 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  - actual_cloned_vm_id = "ID not found or clone failed" -> null
  - cloned_vm_name      = "CloneVM_Terraform" -> null
  - cloned_vm_path      = "C:/Users/dylan/OneDrive/Documents/terraform_debian_lab/CloneVM_Terraform.vmx" -> null
```

```
Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

null_resource.clone_vm: Destroying... [id=4774677232729209 33]
null_resource.clone_vm: Destruction complete after 0s

Destroy complete! Resources: 1 destroyed.
```

Laboratoire de Cybersécurité simple

```
# main.tf

variable "attacker_ip" {
  default = "192.168.159.167"  # IP de la VM attaquante
}

variable "victim_ip" {
  default = "192.168.159.170"  # IP de la VM victime
}

variable "ssh_user" {
  default = "t"
}

variable "ssh_key_path" {
  default = "C:/Users/dylan/.ssh/id_rsa"  # chemin absolu vers ta clé
privée SSH
}

resource "null_resource" "provision_attacker" {
  provisioner "remote-exec" {
    connection {
      type        = "ssh"
      host        = var.attacker_ip
      user        = var.ssh_user
      private_key = file(var.ssh_key_path)
      port        = 22
    }
    inline = [
      "sudo apt update -y",
      "sudo apt install -y nmap masscan"
```

```
    ]
  }
}

resource "null_resource" "provision_victim" {
  provisioner "remote-exec" {
    connection {
      type        = "ssh"
      host        = var.victim_ip
      user        = var.ssh_user
      private_key = file(var.ssh_key_path)
      port        = 22
    }
    inline = [
      "sudo apt update -y",
      "sudo apt install -y python3",
      "nohup python3 -m http.server 8080 &"
    ]
  }
}
```

```
null_resource.provision_victim (remote-exec):   User: t
null_resource.provision_victim (remote-exec):   Password: false
null_resource.provision_victim (remote-exec):   Private key: true
null_resource.provision_victim (remote-exec):   Certificate: false
null_resource.provision_victim (remote-exec):   SSH Agent: false
null_resource.provision_victim (remote-exec):   Checking Host Key: false
null_resource.provision_victim (remote-exec):   Target Platform: unix
null_resource.provision_attacker (remote-exec): Connecting to remote host via SSH...
null_resource.provision_attacker (remote-exec):   Host: 192.168.159.167
null_resource.provision_attacker (remote-exec):   User: t
null_resource.provision_attacker (remote-exec):   Password: false
null_resource.provision_attacker (remote-exec):   Private key: true
null_resource.provision_attacker (remote-exec):   Certificate: false
null_resource.provision_attacker (remote-exec):   SSH Agent: false
null_resource.provision_attacker (remote-exec):   Checking Host Key: false
null_resource.provision_attacker (remote-exec):   Target Platform: unix
null_resource.provision_victim: Still creating... [00m20s elapsed]
null_resource.provision_attacker: Still creating... [00m20s elapsed]
null_resource.provision_victim: Still creating... [00m30s elapsed]
null_resource.provision_attacker: Still creating... [00m30s elapsed]
null_resource.provision_attacker (remote-exec): Connecting to remote host via SSH...
null_resource.provision_attacker (remote-exec):   Host: 192.168.159.167
null_resource.provision_attacker (remote-exec):   User: t
null_resource.provision_attacker (remote-exec):   Password: false
null_resource.provision_attacker (remote-exec):   Private key: true
null_resource.provision_attacker (remote-exec):   Certificate: false
null_resource.provision_attacker (remote-exec):   SSH Agent: false
null_resource.provision_attacker (remote-exec):   Checking Host Key: false
null_resource.provision_attacker (remote-exec):   Target Platform: unix
null_resource.provision_victim (remote-exec): Connecting to remote host via SSH...
null_resource.provision_victim (remote-exec):   Host: 192.168.159.170
null_resource.provision_victim (remote-exec):   User: t
null_resource.provision_victim (remote-exec):   Password: false
null_resource.provision_victim (remote-exec):   Private key: true
null_resource.provision_victim (remote-exec):   Certificate: false
null_resource.provision_victim (remote-exec):   SSH Agent: false
null_resource.provision_victim (remote-exec):   Checking Host Key: false
null_resource.provision_victim (remote-exec):   Target Platform: unix
null_resource.provision_victim: Still creating... [00m40s elapsed]
null_resource.provision_attacker: Still creating... [00m40s elapsed]
```